



Case Técnico – Desenvolvedor Salesforce | Wiz Co

Perguntas técnicas (teoria)

1 - LÓGICA / ARQUITETURA/ CONCEITOS SALESFORCE

- Qual a diferença entre um campo de Pesquisa e um Mestre-Detalhe?
- Qual a diferença de Papel, Perfil e Conjunto de Permissão?
- Uma automação precisa chamar um sistema externo. Qual abordagem você usaria?
 - .Future
 - .Queueable
 - .Platform Event
 - .External Service

Explique sua escolha.

- Um trigger precisa executar um callout HTTP para um sistema externo após a atualização de um registro. Qual abordagem você utilizaria?
- Um processo precisa ser executado todas as noites às 2h da manhã para atualizar registros. Qual abordagem você utilizaria?

2 - APEX – ASSÍNCRONO

- Qual a diferença entre:
 - Batch Apex
 - Queueable Apex
 - Future Methods
- Você pode citar uma situação em que você utilizou Future, Batch e Queueable?

3 - DEBUG / TROUBLESHOOTING

- Um fluxo está falhando em produção, mas funciona no sandbox. Como você investigaria?
- Como usar Debug Logs para encontrar um erro em um trigger?

4 - FLOW (SCREEN FLOW / RECORD TRIGGERED)

- Qual a diferença entre:
 - .Screen Flow
 - .Record-Triggered Flow
 - .Autolaunched Flow
- Quando você escolheria Flow em vez de Apex?



- O que significa um Flow rodar Before Save? Qual a vantagem de um Flow Before Save comparado ao After Save?

5 - API REST / INTEGRAÇÃO

- Qual a diferença entre:
 - .Callout síncrono
 - .Callout assíncrono
- Quando você criaria uma Custom Apex REST API?

6 - EXPERIENCE CLOUD

- O que você sabe sobre Experience Cloud e para que é usado?

Salesforce Developer – Teste prático

Utilizar o Trailhead para desenvolver o teste prático.

Entrega: github + vídeo apresentação (máx.. 15 minutos) ou Zip

Avaliação:

Organização da solução
Boas práticas na plataforma Salesforce
Lógica e modelagem de dados
Uso correto de automações e Apex
Clareza na implementação

1. Modelagem de Dados

Crie os seguintes objetos customizados.

Objeto: PedidoSuporte__c

Campos necessários:

Cliente__c
Tipo: Lookup (Account)

Descricao__c
Tipo: Long Text Area

Status__c
Tipo: Picklist
Valores possíveis:
Novo
Aguardando Cliente
Em Atendimento
Resolvido



Cancelado

Prioridade__c

Tipo: Picklist

Valores possíveis:

Baixa

Média

Alta

UltimaInteracao__c

Tipo: DateTime

Objeto: Interacao__c

Campos necessários:

PedidoSuporte__c

Tipo: Master-Detail (PedidoSuporte__c)

Mensagem__c

Tipo: Long Text Area

Data__c

Tipo: DateTime

Tipo__c

Tipo: Picklist

Valores possíveis:

Cliente

Agente

2. Automação (Flow + Trigger)

2.1 Automação com Flow

Crie um Record-Triggered Flow no objeto Interacao__c.

O Flow deve ser executado quando uma nova interação for criada.

Ao executar, ele deve atualizar o registro relacionado de PedidoSuporte__c com o seguinte valor:

UltimaInteracao__c = Data__c da interação criada.

Além disso, implemente as seguintes regras:

Se Tipo__c = Agente

Atualizar o Status__c do pedido para: Aguardando Cliente



Se Tipo__c = Cliente

Atualizar o Status__c do pedido para: Em Atendimento

Essa automação deve garantir que o pedido seja atualizado automaticamente sempre que uma nova interação for registrada.

2.2 Automação com Trigger (Apex)

Crie um Trigger no objeto PedidoSuporte__c.

O Trigger deve validar a seguinte regra de negócio:

Se um pedido estiver com Prioridade__c = Alta e o status for alterado para Resolvido, o sistema deve verificar se existe pelo menos uma interação do tipo Agente relacionada ao pedido.

Caso não exista nenhuma interação do tipo Agente, o sistema deve impedir a atualização e exibir uma mensagem de erro.

Exemplo de mensagem:

"Pedidos de alta prioridade devem possuir pelo menos uma interação de agente antes de serem resolvidos."

3. Regra de Negócio

Implemente uma regra que não permita cancelar um pedido que já esteja resolvido.

Regra:

Se o Status atual for Resolvido e o usuário tentar alterar para Cancelado, a operação deve ser bloqueada.

4. Processamento Assíncrono (Apex)

Crie uma classe Apex responsável por identificar Pedidos de Suporte que não possuem interação há mais de 3 dias.

Critério de busca:

UltimaInteracao__c menor que NOW() menos 3 dias.

Quando um pedido atender a esse critério, o sistema deve atualizar o status do pedido para:

Aguardando Cliente.



Além disso, deve enviar um e-mail avisando ao cliente que o Pedido de Suporte será fechado em breve devido a inatividade. (Considerar o e-mail da conta)

5. API REST

Crie uma Apex REST API que permite criar novos pedidos de suporte.

Endpoint esperado:

POST /services/apexrest/pedidos-suporte

Exemplo de payload recebido pela API:

```
{
  "clienteld": "001XXXXXXXXXXXXX",
  "descricao": "Problema no sistema",
  "prioridade": "Alta"
}
```

A API deve realizar as seguintes ações:

Criar um registro PedidoSuporte__c.

Definir automaticamente os seguintes valores:

Status__c = Novo

UltimaInteracao__c = NOW()

A resposta da API deve retornar o identificador do pedido criado.

Exemplo de resposta:

```
{
  "pedidoid": "a01XXXXXXXXXXXXX"
}
```

6. Experience Cloud

Crie uma Community nessa etapa.

Crie um Screen Flow que permita a um cliente:

Criar um pedido de suporte.

Adicionar uma interação ao pedido.

Esse Flow deve ser estruturado de forma que possa ser facilmente exposto em uma página do Experience Cloud. O fluxo deve ser invocado em um botão na página da Conta, permitindo já vincular a conta ao pedido.